differentiating features 85–86
domain model 268–269
Domain-Driven Design. *See* DDD

E

embedded tables for Behave 168–169
@end-to-end tag 189
environment.py module 189
epics 99–100
examples
  conversations about 111–113
  illustrated 90
  illustrating features with 100–106
  turning into executable specifications 115–118
  understanding features using 67
  See also scenarios
exclamation point (!) 124
executable scenarios. *See* scenarios
executable specifications
  in automated build process
    overview 322
    running from command line 325
    self-sufficient specifications 322–323
    using version control 324–325
  defined 44
  implementing features 42–46
  as living documentation
    fluent assertions in JavaScript 295
    fluent assertions in static languages 295–297
    fluent coding 294
  principles of BDD 21–23
  turning examples into 115–118
ExpectedConditions class 220–221
Expeditionary Combat Support System 3
explicit waits 220

F

failure of software projects 4
feature files
  for Cucumber-JS 175–176
  description for 119–120
  organizing 136
  for SpecFlow 170–171
  storing scenarios in 135–136
features
  breaking into manageable pieces 93–95
  breaking into stories 37
  capabilities and 90–93
  conversations about examples 111–113
  coverage of requested 305–308
  defined 16, 67, 87
  Deliberate Discovery 110–111
  delivering business value 16–17
  describing 35–37
  differentiating features 85–86
  illustrated 89
  illustrating stories with examples 38
  illustrating with concrete examples 18–20
  illustrating with examples 100–106
  implementing
    automated tests 46–55
    executable specifications 42–46
    going from examples to acceptance criteria 39–40
    setting up Maven and Git 40–41
    tests as living documentation 55–56
  injecting
    defined 62
    gathering information through 65–66
    hunting business value 64–65
    overview 63–64, 66–67
    using those with most value 65
  minimum impact features 86
  overview 88–90
  parity features 86
  partner features 86
  providing highest ROI 84–86
  readiness of 304–305
  Real Options principle
    delaying until last responsible moment 110
    options expire 109
    options have value 108–109
    overview 107–108
  specifying 17
  user stories and
    describing features using 95–98
    differences between 98–99
    epics 99–100
    features covering multiple 100
feedback and tests 185
FEST-Assert library 215, 295
@FindBy annotation 224–225
findElement() method 210, 217
Firefox 210
FitNesse 314
five-why technique 74
fluent assertions
  in JavaScript 295
  in static languages 295–297
  .Net Fluent Assertions library 295–297
fluent coding 294
fluent selectors 234
FluentWait class 221
Flying High Airlines example 63
<forkCount> option 334
Frequent Flyer website 208
Freshen 167
Function interface 221

G

Geb 233
getAllSelectedOptions() method 219
getAttribute() method 219
getAttributeValue() method 218
getFirstSelectedOption() method 219
getters/setters 272
getText() method 215, 218–219
Gherkin 19, 122, 142, 163, 167
Git 40–41
GitHub 40
Given … When … Then structure 52, 122, 129–130
@Given annotation 47, 156
given keyword 39
[Given] attribute 172
GivenStories keyword 133